UNIVERSITAS BENGKULU — FAKULTAS TEKNIK

Prodi S1 Informatika

JAWABAN PROBLEM SET MODUL MINGGU KE-3

Teknologi Internet of Things (IoT)

Firmware ESP32, Bus I2C/SPI, FreeRTOS, dan Deep Sleep

Nama / NPM anggota	1. Tasya Triani G1A024015 2. Nayla Viona Azzarah G1A024045 3. Angelita Rahmatun annisa G1A024067 4. Tri Haiji Januarli G1A024069

Dokumen ini berisi jawaban enam butir soal berjenjang pada Bagian 6 Modul Minggu 3, dari level C1 (Mengingat) hingga C6 (Mencipta). Setiap soal ditulis ulang, lalu dijawab langkah demi langkah. Dokumen ini terpisah dari Laporan Praktikum Lab-2.

Soal 1 — Level C1 (Mengingat)

> **Soal:** (a) Sebutkan fungsi masing-masing jalur sinyal pada antarmuka bus I2C dan SPI! (b) Berapakah alamat I2C baku untuk sensor BME280 dan layar OLED SSD1306? (c) Sebutkan lima mode daya pada mikrokontroler ESP32 dari yang mengonsumsi arus terbesar hingga terkecil!

Jawaban (a): Fungsi jalur sinyal I2C dan SPI

I2C (2 jalur sinyal)

Jalur	Fungsi
SDA (Serial Data)	Jalur data dua arah (bidirectional). Dipakai master dan slave bergantian untuk mengirim alamat, perintah, dan data. Pin default ESP32: GPIO 21.
SCL (Serial Clock)	Jalur detak (clock) yang dihasilkan master untuk menyinkronkan pengambilan bit pada SDA. Pin default ESP32: GPIO 22.

Seluruh peranti berbagi dua jalur yang sama. Setiap slave dibedakan oleh alamat 7-bit. Jalur bersifat open-drain sehingga membutuhkan resistor pull-up (±2,2–4,7 kΩ) ke 3,3 V.

SPI (4 jalur sinyal)

Jalur	Fungsi
MOSI (Master Out Slave In)	Data dari master ke slave.
MISO (Master In Slave Out)	Data dari slave ke master.

Jalur	Fungsi
SCK (Serial Clock)	Detak sinkronisasi dari master.
CS / SS (Chip Select)	Pemilih peranti. Master menarik CS slave target ke LOW untuk mengaktifkannya. Dibutuhkan satu pin CS per slave.

SPI bersifat full-duplex dan sangat cepat (hingga 40–80 MHz), tanpa pengalamatan di dalam paket data.

Jawaban (b): Alamat I2C baku

Peranti	Alamat baku	Keterangan
BME280 (juga BMP280)	0x76 atau 0x77	Ditentukan pin SDO: SDO ke GND = 0x76, SDO ke VCC (3,3 V) = 0x77.
OLED SSD1306	0x3C (kadang 0x3D)	Alamat 0x3C paling umum pada modul 0,96 inci; sebagian modul dapat diubah ke 0x3D.

Jawaban (c): Lima mode daya ESP32 (terbesar ke terkecil)

No	Mode daya	Arus tipikal	Keterangan singkat
1	Active Mode	80–240 mA	CPU ganda, radio Wi-Fi/BT, dan seluruh periferal aktif.
2	Modem Sleep	20–30 mA	CPU aktif, radio dimatikan (hanya hidup periodik).
3	Light Sleep	0,8–2,0 mA	Clock CPU dijeda, RAM dipertahankan, eksekusi dilanjutkan setelah bangun.
4	Deep Sleep	10–150 µA	CPU dan RAM utama mati; hanya RTC dan ULP hidup. Bangun = reboot.
5	Power Down	≈ 5 µA	Seluruh keping dimatikan lewat pin CHIP_PU (EN).

Soal 2 — Level C2 (Memahami)

> **Soal:** Jelaskan secara mendalam: (a) Mengapa bus I2C menggunakan konfigurasi rangkaian open-drain dan membutuhkan resistor pull-up eksternal? Apa yang terjadi pada jalur SCL/SDA jika resistor pull-up tidak terpasang? (b) Jelaskan perbedaan mendasar siklus

> eksekusi kode program saat ESP32 terbangun dari mode Light Sleep dibandingkan terbangun dari Deep Sleep!

Jawaban (a): Open-drain dan resistor pull-up

Alasan penggunaan open-drain

- Keamanan listrik pada bus bersama. Banyak peranti (bahkan banyak master) terhubung ke dua kabel yang sama. Bila keluaran berupa push-pull, satu peranti yang mengeluarkan HIGH dan peranti lain yang mengeluarkan LOW pada saat bersamaan akan menciptakan hubung singkat antara VCC dan GND.
- Cara kerja open-drain. Setiap peranti hanya dapat menarik jalur ke GND (transistor ON, logika LOW) atau melepasnya (transistor OFF, high-impedance). Tidak ada peranti yang pernah mendorong jalur ke VCC, sehingga tidak ada risiko hubung singkat.
- Wired-AND. Jalur bernilai HIGH hanya jika semua peranti melepasnya; jika salah satu menarik ke GND, jalur menjadi LOW. Sifat ini dipakai untuk arbitrase multi-master, ACK, dan clock stretching (slave menahan SCL LOW agar master menunggu).

Peran resistor pull-up

Karena tidak ada peranti yang menarik jalur ke HIGH, kondisi HIGH (idle) harus dihasilkan oleh resistor pull-up ke 3,3 V, umumnya 2,2–4,7 kΩ. Nilai resistor menentukan kecepatan naik sinyal (rise time) bersama kapasitansi bus Cb:

> t_rise (30% -> 70% VDD) ≈ 0,8473 x Rp x Cb
>
> Contoh Cb = 100 pF:
> Rp = 4,7 kΩ -> t_rise ≈ 0,8473 x 4700 x 100e-12 ≈ 398 ns (layak untuk 100 kHz, batas 1000 ns)
> Rp = 2,2 kΩ -> t_rise ≈ 0,8473 x 2200 x 100e-12 ≈ 186 ns (layak untuk 400 kHz, batas 300 ns)

Resistor terlalu besar membuat sinyal naik lambat (bentuk gelombang membulat, data rusak pada kecepatan tinggi). Resistor terlalu kecil membuat arus saat LOW terlalu besar dan melebihi kemampuan sink pin.

Jika pull-up tidak terpasang

- Jalur SDA/SCL tidak punya sumber tegangan HIGH sehingga mengambang (floating) dan tidak pernah mencapai level HIGH yang valid.

- Peranti hanya bisa menarik ke LOW; saat melepas, jalur tidak kembali HIGH sehingga clock dan data tidak terbentuk benar.
- Akibatnya: tidak ada ACK dari slave, I2C Scanner tidak menemukan peranti (0 peranti), atau data rusak oleh derau. Pada praktik, banyak modul breakout sudah membawa pull-up bawaan, namun jangan lupa pada rangkaian sendiri.

Jawaban (b): Light Sleep vs Deep Sleep

Aspek	Light Sleep	Deep Sleep
Keadaan CPU	Clock dijeda (clock gating), konteks CPU tersimpan	CPU dimatikan total
RAM utama (SRAM)	Dipertahankan (daya tetap)	Hilang (daya diputus)
Memori yang bertahan	Semua variabel dan stack	Hanya RTC Slow Memory (RTC_DATA_ATTR)
Arus tipikal	0,8–2,0 mA	10–150 µA
Perilaku saat bangun	Melanjutkan eksekusi tepat pada baris setelah esp_light_sleep_start()	Reboot dari awal; setup() dieksekusi kembali, setelah bootloader
Variabel biasa	Nilai tetap utuh	Direset ke nilai awal
Cocok untuk	Menunggu singkat (< 1 detik) dengan konteks dipertahankan	Interval panjang (menit–jam) demi umur baterai maksimum

Kesimpulan: Light Sleep berperilaku seperti jeda (pause), sedangkan Deep Sleep berperilaku seperti mematikan lalu menyalakan ulang perangkat. Karena itu pada Deep Sleep, kode inisialisasi harus dirancang untuk dijalankan berulang dan data lintas siklus wajib disimpan di memori RTC.

Soal 3 — Level C3 (Menerapkan)

> **Soal:** Sebuah stasiun pengukur cuaca di lereng Gunung Patah Bengkulu mencatat tekanan atmosfer sebesar 895,2 hPa. (a) Dengan asumsi tekanan standar permukaan laut P0 = 1013,25 hPa, hitunglah estimasi ketinggian stasiun tersebut di atas permukaan laut! (b) Jika simpul tersebut ditenagai baterai Li-Ion kapasitas 2500 mAh (η = 0,85), aktif selama 2 detik (I_aktif

> = 90 mA) untuk membaca I2C dan tidur Deep Sleep selama 10 menit (I_tidur = 30 µA), hitunglah konsumsi arus rata-rata I_avg dan estimasi masa pakai baterai dalam satuan hari!

Jawaban (a): Ketinggian barometrik

Rumus: h = 44330 × [1 − (P / P0)^(1/5,255)]

1. Hitung rasio tekanan: P / P0 = 895,2 / 1013,25 = 0,88349
2. Hitung pangkat: 1 / 5,255 = 0,190295
3. Hitung (0,88349)^0,190295. Dengan logaritma: ln(0,88349) = −0,12388; dikalikan 0,190295 = −0,023574; eksponen: e^(−0,023574) = 0,97670
4. Hitung selisih: 1 − 0,97670 = 0,02330
5. Hitung ketinggian: h = 44330 × 0,02330 ≈ 1.033 m

Hasil (a): ketinggian stasiun ≈ 1.033 m di atas permukaan laut. Nilai ini wajar untuk lereng gunung. Perlu diingat bahwa estimasi ini mengasumsikan atmosfer standar; tekanan permukaan laut aktual yang berubah menurut cuaca dapat menggeser hasil puluhan meter.

Jawaban (b): Arus rata-rata dan masa pakai baterai

Diketahui: t_aktif = 2 s, I_aktif = 90 mA; t_tidur = 10 menit = 600 s, I_tidur = 30 µA = 0,030 mA; kapasitas 2500 mAh; η = 0,85.

> Periode siklus T = t_aktif + t_tidur = 2 + 600 = 602 s
>
> I_avg = (I_aktif x t_aktif + I_tidur x t_tidur) / T
> = (90 x 2 + 0,030 x 600) / 602
> = (180 + 18) / 602
> = 198 / 602
> = 0,3289 mA
>
> Kapasitas efektif = 2500 mAh x 0,85 = 2125 mAh
> Masa pakai (jam) = 2125 / 0,3289 = 6.461 jam
> Masa pakai (hari) = 6.461 / 24 = 269,2 hari (± 8,9 bulan)

Hasil (b): I_avg ≈ 0,329 mA dan masa pakai ≈ 269 hari (sekitar 9 bulan).

Catatan: sekitar 91% energi terpakai pada fase aktif (180 dari 198 mA·s), padahal hanya 0,33% waktu. Memperpendek waktu aktif atau menurunkan arus aktif jauh lebih efektif daripada menurunkan arus tidur. Self-discharge baterai dan arus regulator tidak dihitung.

Soal 4 — Level C4 (Menganalisis)

> **Soal:** Seorang mahasiswa Informatika mengeluhkan bahwa saat ia menampilkan animasi grafik yang rumit pada layar OLED SSD1306 di dalam fungsi loop(), pembacaan sensor cuaca menjadi tersendat dan pembacaan interupsi tombol sering terlewat. Di samping itu, variabel pencacah paket int counter = 0; selalu kembali bernilai 0 setiap kali modul ESP32 bangun dari Deep Sleep. Analisislah dua kelemahan arsitektur perangkat lunak tersebut dan berikan solusi perbaikan kode konkretnya menggunakan fitur FreeRTOS dan memori RTC!

Analisis kelemahan

Gejala	Akar masalah	Solusi
Pembacaan sensor tersendat dan tombol sering terlewat	Animasi OLED berjalan di loop() dan bersifat blocking. Penggambaran buffer ditambah display.display() mengirim 1024 byte lewat I2C 400 kHz (±25–30 ms per frame). Selama itu CPU tidak sempat memanggil pembacaan sensor atau memeriksa tombol (polling). Semua pekerjaan tersusun serial dalam satu alur.	Pisahkan pekerjaan menjadi task FreeRTOS: task OLED di Core 0 dengan vTaskDelay, sedangkan sensor dan logika utama di Core 1. Tombol ditangani dengan hardware interrupt (ISR bertanda IRAM_ATTR) yang hanya mengubah flag volatile.
counter selalu kembali 0 setelah bangun dari Deep Sleep	int counter = 0; ditempatkan di SRAM biasa. Saat Deep Sleep SRAM kehilangan daya; saat bangun terjadi reboot dan variabel diinisialisasi ulang ke 0.	Tempatkan di RTC Slow Memory dengan atribut RTC_DATA_ATTR agar nilainya bertahan selama tidur.

Solusi kode

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_SSD1306.h>

Adafruit_SSD1306 display(128, 64, &Wire, -1);

RTC_DATA_ATTR int counter = 0;           // (2) bertahan saat Deep Sleep
volatile bool flagTombol = false;        // flag dari ISR harus volatile
volatile unsigned long tIsrTerakhir = 0;
SemaphoreHandle_t mutexI2C;              // melindungi bus I2C bersama

// (1) ISR singkat: hanya set flag, tanpa delay/I2C/Serial
void IRAM_ATTR isrTombol() {
  unsigned long now = millis();
  if (now - tIsrTerakhir > 50) {         // debouncing 50 ms
    flagTombol = true;
    tIsrTerakhir = now;
  }
}

// Task OLED di Core 0: animasi tidak memblokir sensor/tombol
void TaskOled(void *pv) {
  int frame = 0;
  for (;;) {
    if (xSemaphoreTake(mutexI2C, pdMS_TO_TICKS(100)) == pdTRUE) {
      display.clearDisplay();
      display.setTextSize(1);
      display.setTextColor(SSD1306_WHITE);
      display.setCursor(0, 0);
      display.printf("Paket #%d  frame %d", counter, frame++);
      display.fillCircle((frame * 4) % 128, 40, 6, SSD1306_WHITE); // animasi
      display.display();                 // ±25-30 ms, tapi di Core 0
      xSemaphoreGive(mutexI2C);
```

```
    }
    vTaskDelay(pdMS_TO_TICKS(100));       // non-blocking: beri waktu task lain
  }
}

void setup() {
  Serial.begin(115200);
  counter++;                              // naik 1 setiap bangun (tidak reset)
  Serial.printf("Siklus ke-%d\n", counter);

  mutexI2C = xSemaphoreCreateMutex();
  Wire.begin(21, 22);
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);

  pinMode(15, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(15), isrTombol, FALLING);

  // Core 0: OLED. Core 1: loop() untuk sensor & logika.
  xTaskCreatePinnedToCore(TaskOled, "OLED", 4096, NULL, 1, NULL, 0);
}

void loop() {                             // Core 1
  if (flagTombol) {                       // respons tombol hampir seketika
    flagTombol = false;
    Serial.println("Tombol darurat ditekan!");
  }
  // bacaSensor(): jika memakai I2C, ambil mutexI2C dulu
  vTaskDelay(pdMS_TO_TICKS(50));
}
```

Penjelasan perbaikan:

- Animasi dipindah ke TaskOled di Core 0 sehingga display.display() (±25–30 ms) tidak lagi menahan loop() di Core 1.
- ISR hanya menaikkan flag volatile; tombol tidak lagi bergantung pada kecepatan polling dan tidak terlewat.

- mutexI2C mencegah dua task mengakses bus I2C bersamaan (pustaka Wire tidak thread-safe).
- RTC_DATA_ATTR membuat counter bertahan melewati Deep Sleep; pada boot pertama (power-on) nilainya 0, lalu 1, 2, 3, dst.

Soal 5 — Level C5 (Mengevaluasi)

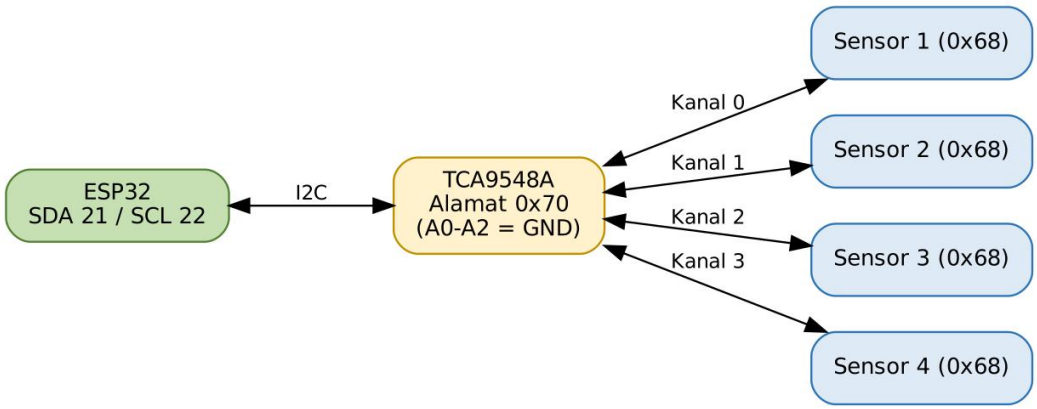
> **Soal:** Sebuah tim pengembang IoT mengusulkan sistem pemantauan gempa bumi dan pergerakan tanah di pesisir Bengkulu dengan menghubungkan 4 sensor akselerometer I2C yang identik (part number sama dengan alamat baku tunggal 0x68 tanpa pin pengubah alamat SDO). Evaluasilah kelayakan usulan tersebut jika seluruh sensor dipasang pada bus I2C yang sama! Bandingkan tiga alternatif solusi rekayasanya: (1) Menggunakan chip I2C Multiplexer (TCA9548A), (2) Menggunakan bus perangkat lunak (Software Bit-Banged I2C), atau (3) Memanfaatkan periferal antarmuka I2C ganda (Wire dan Wire1) pada ESP32!

1. Evaluasi kelayakan usulan awal

Kesimpulan: usulan tidak layak bila keempat sensor dipasang langsung pada satu bus I2C.

- Pengalamatan I2C bergantung pada alamat 7-bit yang unik. Keempat sensor memakai 0x68 tanpa pin SDO/ADDR, sehingga alamatnya tidak dapat diubah.
- Saat master memanggil 0x68, keempat sensor merespons bersamaan; ACK dan data saling menimpa (wired-AND), sehingga data yang terbaca rusak atau tidak dapat diandalkan, dan master tidak bisa memilih sensor tertentu.
- Pada aplikasi gempa, data yang salah berarti peringatan palsu atau gempa tidak terdeteksi, sehingga risikonya tinggi.

2. Perbandingan tiga alternatif



Gambar 1. Topologi alternatif 1: empat sensor beralamat sama melalui TCA9548A

Aspek	(1) TCA9548A	(2) I2C perangkat lunak	(3) Wire + Wire1
Prinsip	Chip multiplexer 8 kanal; master memilih satu kanal aktif (register kontrol) sebelum berbicara dengan sensor	I2C ditiru lewat bit-banging GPIO, masing-masing sensor memakai pasangan pin SDA/SCL sendiri	Memakai dua controller I2C hardware ESP32 (I2C0 dan I2C1), masing-masing satu bus
Jumlah sensor beralamat sama	Hingga 8 (bahkan 64 dengan 8 chip)	Sebanyak pasangan pin yang tersedia (4 sensor = 8 pin)	Hanya 2 (satu per bus)
Kinerja	Kecepatan hardware penuh (hingga 400 kHz); ada latensi kecil saat ganti kanal	Lambat dan jitter; memakan waktu CPU; terganggu interupsi dan Wi-Fi	Cepat, hardware murni, tanpa beban CPU berarti
Biaya dan kompleksitas	Tambah 1 chip kecil + kode pemilih kanal; murah	Tanpa perangkat keras tambahan, tapi kode dan pin banyak	Tanpa perangkat keras tambahan; kode sederhana (Wire1.begin)
Keandalan untuk sampling gempa	Baik; pembacaan bergantian per sensor	Rendah; timing tidak presisi	Baik, tetapi jumlah sensor tidak cukup
Kelayakan untuk 4 sensor	Layak	Layak secara teknis, kurang cocok	Tidak cukup (maksimum 2)

3. Penilaian berbobot

Kriteria	Bobot	TCA9548A	Software I2C	Wire + Wire1
Kemampuan melayani 4 sensor	30%	5	4	1
Presisi/kecepatan sampling	25%	4	2	5
Keandalan dan stabilitas	20%	4	2	5
Kemudahan implementasi	15%	4	3	5

Kriteria	Bobot	TCA9548A	Software I2C	Wire + Wire1
Biaya dan pin yang dipakai	10%	4	2	5
Total skor (maks. 5)	100%	4,30	2,75	3,80

Skor 1–5 adalah penilaian subjektif penulis, dipakai untuk memperlihatkan alur evaluasi; bobot dapat disesuaikan kebutuhan proyek. Contoh perhitungan (1): 0,30×5 + 0,25×4 + 0,20×4 + 0,15×4 + 0,10×4 = 1,50 + 1,00 + 0,80 + 0,60 + 0,40 = 4,30.

4. Rekomendasi

Gunakan alternatif (1) TCA9548A: satu bus hardware, delapan kanal, mudah diskalakan, dan kecepatan penuh. Bila diperlukan pembacaan lebih cepat atau redundansi, kombinasikan: Wire dengan satu TCA9548A (2 sensor) dan Wire1 dengan TCA9548A kedua. Catatan teknis:

- Alamat TCA9548A berada di 0x70–0x77, sehingga tidak bentrok dengan 0x68.
- Sensor tidak dibaca benar-benar serentak, melainkan bergantian; perhitungkan selisih waktu antar kanal (beri stempel waktu bila perlu sinkronisasi antar sensor).
- Pada setiap kanal tetap perlu pull-up, dan kapasitansi bus per kanal dijaga di bawah 400 pF (kabel pendek) terutama di lingkungan pesisir yang lembap dan korosif.
- Bila memakai FreeRTOS, lindungi seluruh akses bus I2C dengan mutex agar pemilihan kanal dan pembacaan tidak terpotong task lain.

```
#define TCA_ADDR 0x70
#define ADDR_AKSEL 0x68

void tcaPilih(uint8_t kanal) {          // aktifkan satu kanal TCA9548A
  if (kanal > 7) return;
  Wire.beginTransmission(TCA_ADDR);
  Wire.write(1 << kanal);               // bit kanal = 1
  Wire.endTransmission();
}

void bacaSemuaSensor() {
  for (uint8_t k = 0; k < 4; k++) {
    tcaPilih(k);                        // sensor k kini satu-satunya di bus
    Wire.beginTransmission(ADDR_AKSEL);
    // ... tulis register data, baca X/Y/Z akselerometer ...
```

```
  Wire.endTransmission();
 }
}
```

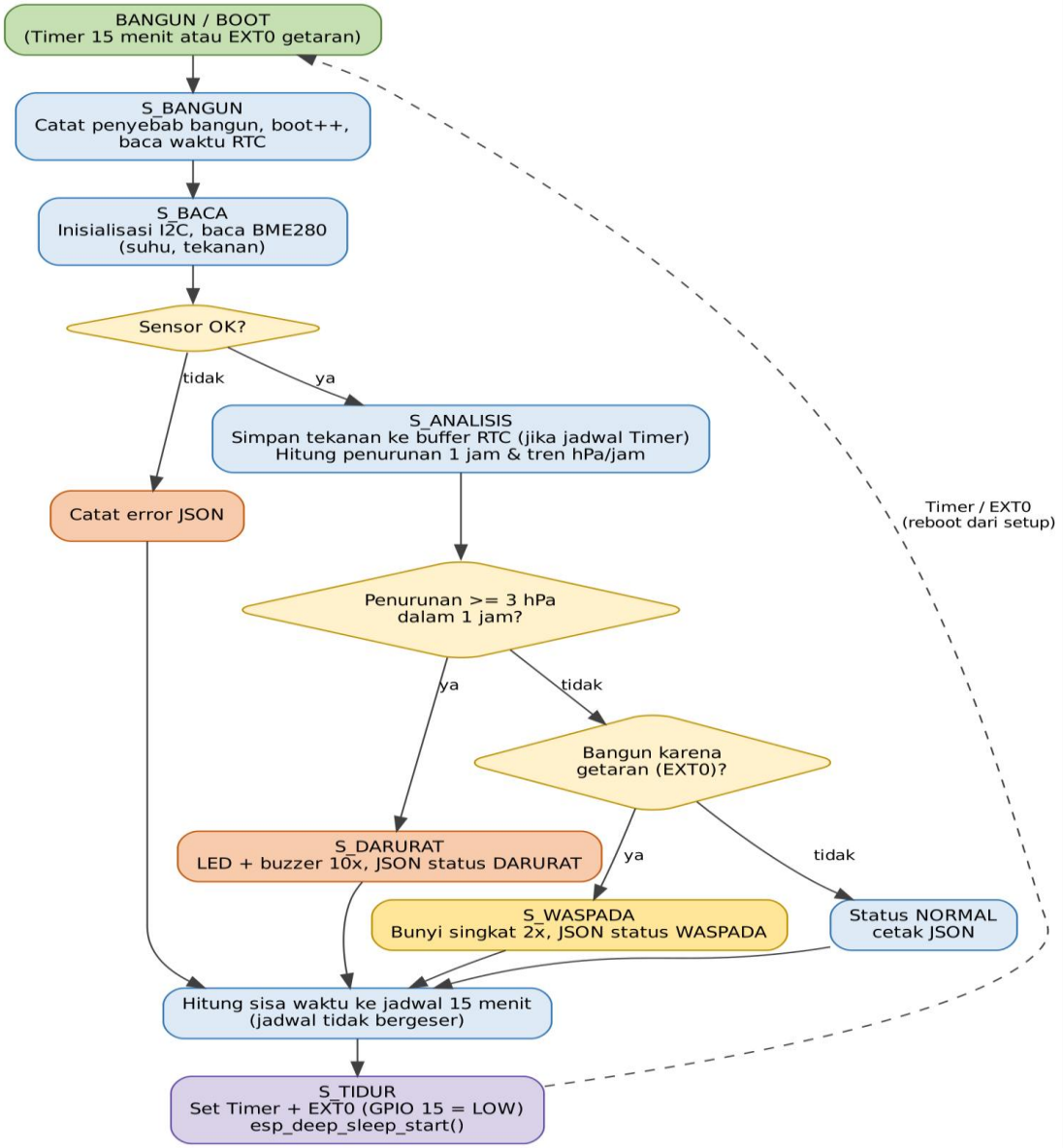
Soal 6 — Level C6 (Mencipta)

Soal: Rancanglah diagram alir arsitektur perangkat lunak (software state machine) dan susunlah program C++ lengkap untuk sistem "Smart Early-Warning Landslide Node" yang mengintegrasikan: (a) Pengambilan data sensor BME280 via I2C setiap 15 menit menggunakan Timer Deep Sleep, (b) Sensor getaran fisis yang memicu pin interupsi EXT0 Wakeup jika terjadi guncangan tiba-tiba sebelum jadwal timer berakhir, (c) Penyimpanan 10 data riwayat tekanan udara terakhir di dalam memori RTC_DATA_ATTR untuk menghitung tren perubahan cuaca, dan (d) Pengaktifan sinyal darurat jika terjadi penurunan tekanan drastis ≥ 3 hPa dalam 1 jam!

1. Analisis kebutuhan dan keputusan desain

Butir	Kebutuhan	Keputusan desain
(a)	BME280 dibaca tiap 15 menit	esp_sleep_enable_timer_wakeup(sisa waktu); interval INTERVAL_S = 900 detik. Sisa waktu dihitung ulang agar jadwal tidak bergeser meski node dibangunkan getaran.
(b)	Getaran membangunkan node sebelum jadwal	esp_sleep_enable_ext0_wakeup(GPIO_NUM_15, 0). Sensor getar dianggap normal HIGH (pull-up) dan LOW saat bergetar.
(c)	Riwayat 10 data tekanan di RTC	Buffer melingkar (circular buffer) rtcTekanan[10] beserta stempel waktu rtcWaktu[10], keduanya RTC_DATA_ATTR.
(d)	Darurat bila turun ≥ 3 hPa dalam 1 jam	Penurunan = tekanan tertinggi dalam jendela 3600 s dikurangi tekanan sekarang; jika ≥ 3,0 hPa maka S_DARURAT (LED, buzzer, dan JSON ALERT).

2. Diagram alir (state machine)



Gambar 2. Diagram alir state machine Smart Early-Warning Landslide Node

State	Aksi	Transisi berikutnya
S_BANGUN	Baca penyebab bangun (Timer/EXT0/boot), tambah penghitung boot, baca waktu RTC	S_BACA
S_BACA	Inisialisasi I2C dan BME280, ukur suhu dan tekanan (mode forced)	Sensor OK → S_ANALISIS; gagal → catat error lalu S_TIDUR

State	Aksi	Transisi berikutnya
S_ANALISIS	Simpan tekanan ke buffer RTC (khusus bangun terjadwal), hitung penurunan 1 jam dan tren hPa/jam	Turun ≥ 3 hPa → S_DARURAT; bangun getaran → S_WASPADA; lainnya → S_TIDUR (NORMAL)
S_DARURAT	LED dan buzzer berkedip 10×, cetak JSON status DARURAT	S_TIDUR
S_WASPADA	Bunyi singkat 2×, cetak JSON status WASPADA	S_TIDUR
S_TIDUR	Hitung sisa waktu ke jadwal, set Timer dan EXT0, esp_deep_sleep_start()	Bangun (Timer/EXT0) → reboot ke S_BANGUN

3. Program lengkap (C++ / Arduino-ESP32)

```
#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>
#include <sys/time.h>
#include "esp_sleep.h"
#include "driver/rtc_io.h"

// ---------------- Konfigurasi ----------------
#define PIN_GETARAN   GPIO_NUM_15   // sensor getar: normal HIGH, getar -> LOW
#define PIN_LED       2
#define PIN_BUZZER    4
#define INTERVAL_S    900           // 15 menit
#define N_RIWAYAT     10            // jumlah data riwayat tekanan
#define AMBANG_DROP   3.0f          // hPa
#define JENDELA_S     3600          // 1 jam

// ---------------- Memori RTC (bertahan saat Deep Sleep) ----------------
RTC_DATA_ATTR float    rtcTekanan[N_RIWAYAT];
```

```
RTC_DATA_ATTR uint32_t rtcWaktu[N_RIWAYAT];
RTC_DATA_ATTR uint8_t  rtcJumlah = 0;   // banyak data valid (0..10)
RTC_DATA_ATTR uint8_t  rtcIdx    = 0;   // posisi tulis berikutnya
RTC_DATA_ATTR uint32_t rtcJadwal = 0;   // waktu jadwal Timer berikutnya (detik)
RTC_DATA_ATTR uint32_t rtcBoot   = 0;   // penghitung siklus bangun

Adafruit_BME280 bme;

enum Keadaan { S_BANGUN, S_BACA, S_ANALISIS, S_DARURAT, S_WASPADA,
S_TIDUR };

uint32_t sekarangDetik() {              // waktu RTC tetap berjalan saat tidur
  struct timeval tv;
  gettimeofday(&tv, NULL);
  return (uint32_t)tv.tv_sec;
}

// Simpan ke buffer melingkar: data terlama ditimpa saat penuh
void simpanTekanan(float p, uint32_t t) {
  rtcTekanan[rtcIdx] = p;
  rtcWaktu[rtcIdx]   = t;
  rtcIdx = (rtcIdx + 1) % N_RIWAYAT;
  if (rtcJumlah < N_RIWAYAT) rtcJumlah++;
}

// (d) Penurunan = tekanan tertinggi dalam 1 jam terakhir - tekanan sekarang
float hitungPenurunan(float pNow, uint32_t tNow) {
  float pMaks = pNow;
  for (uint8_t i = 0; i < rtcJumlah; i++)
    if ((tNow - rtcWaktu[i]) <= JENDELA_S && rtcTekanan[i] > pMaks)
      pMaks = rtcTekanan[i];
  return pMaks - pNow;
}
```

```
// (c) Tren cuaca (hPa/jam): selisih data terbaru vs terlama pada buffer
float hitungTren(float pNow, uint32_t tNow) {
  if (rtcJumlah == 0) return 0.0f;
  uint8_t tertua = (rtcJumlah < N_RIWAYAT) ? 0 : rtcIdx;
  uint32_t dt = tNow - rtcWaktu[tertua];
  if (dt < 60) return 0.0f;
  return (pNow - rtcTekanan[tertua]) / (dt / 3600.0f);
}

void alarmBunyi(int kali, int ms) {
  for (int i = 0; i < kali; i++) {
    digitalWrite(PIN_LED, HIGH); digitalWrite(PIN_BUZZER, HIGH); delay(ms);
    digitalWrite(PIN_LED, LOW);  digitalWrite(PIN_BUZZER, LOW);  delay(ms);
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_GETARAN, INPUT_PULLUP);

  Keadaan s = S_BANGUN;
  esp_sleep_wakeup_cause_t sebab = ESP_SLEEP_WAKEUP_UNDEFINED;
  bool bangunGetaran = false, bangunJadwal = false;
  uint32_t now = 0;
  float suhu = NAN, tekanan = NAN, drop = 0, tren = 0;
  const char *status = "NORMAL";
  uint32_t sisa = INTERVAL_S;

  while (true) {
    switch (s) {

      case S_BANGUN: {
```

```
  rtcBoot++;
  sebab = esp_sleep_get_wakeup_cause();
  bangunGetaran = (sebab == ESP_SLEEP_WAKEUP_EXT0);   // (b)
  bangunJadwal  = (sebab == ESP_SLEEP_WAKEUP_TIMER);  // (a)
  now = sekarangDetik();
  s = S_BACA;
  break;
 }

 case S_BACA: {
  Wire.begin(21, 22);
  if (bme.begin(0x76)) {
   bme.setSampling(Adafruit_BME280::MODE_FORCED,
                   Adafruit_BME280::SAMPLING_X2,
                   Adafruit_BME280::SAMPLING_X16,
                   Adafruit_BME280::SAMPLING_X1,
                   Adafruit_BME280::FILTER_X16);
   bme.takeForcedMeasurement();
   suhu    = bme.readTemperature();
   tekanan = bme.readPressure() / 100.0f;          // Pa -> hPa
   s = S_ANALISIS;
  } else {
   Serial.println("{\"error\":\"BME280 tidak terdeteksi\"}");
   s = S_TIDUR;
  }
  break;
 }

 case S_ANALISIS: {
  drop = hitungPenurunan(tekanan, now);
  tren = hitungTren(tekanan, now);
  // Riwayat hanya diisi pada boot pertama atau bangun terjadwal,
  // agar interval data konsisten (15 menit)
  if (bangunJadwal || rtcJumlah == 0) simpanTekanan(tekanan, now);
```

```
    if (drop >= AMBANG_DROP)  { s = S_DARURAT; status = "DARURAT"; }
    else if (bangunGetaran)   { s = S_WASPADA; status = "WASPADA"; }
    else                      { s = S_TIDUR;   status = "NORMAL";  }

    Serial.printf("{\"boot\":%u,\"cause\":\"%s\",\"temp_c\":%.2f,"
                  "\"pressure_hpa\":%.2f,\"drop_1h_hpa\":%.2f,"
                  "\"trend_hpa_per_h\":%.2f,\"status\":\"%s\"}\n",
                  rtcBoot,
                  bangunGetaran ? "GETARAN" : (bangunJadwal ? "TIMER" : "BOOT"),
                  suhu, tekanan, drop, tren, status);
    break;
  }

  case S_DARURAT:                                  // (d)
    Serial.println("{\"ALERT\":\"Penurunan tekanan >= 3 hPa dalam 1 jam\"}");
    alarmBunyi(10, 250);
    s = S_TIDUR;
    break;

  case S_WASPADA:
    alarmBunyi(2, 150);
    s = S_TIDUR;
    break;

  case S_TIDUR: {
    // Pertahankan jadwal 15 menit meskipun dibangunkan getaran
    if (bangunJadwal || sebab == ESP_SLEEP_WAKEUP_UNDEFINED || now >=
rtcJadwal)
      rtcJadwal = now + INTERVAL_S;
    sisa = (rtcJadwal > now) ? (rtcJadwal - now) : 1;

    // Tunggu sensor getar kembali HIGH agar tidak langsung bangun lagi
    uint32_t t0 = millis();
```

```
      while (digitalRead(PIN_GETARAN) == LOW && millis() - t0 < 3000) delay(10);

      rtc_gpio_pullup_en(PIN_GETARAN);
      rtc_gpio_pulldown_dis(PIN_GETARAN);
      esp_sleep_enable_timer_wakeup((uint64_t)sisa * 1000000ULL);  // (a)
      esp_sleep_enable_ext0_wakeup(PIN_GETARAN, 0);              // (b)

      digitalWrite(PIN_LED, LOW);
      digitalWrite(PIN_BUZZER, LOW);
      Serial.printf("Tidur %u detik...\n", sisa);
      Serial.flush();
      esp_deep_sleep_start();                    // tidak kembali
      break;
    }
   }
  }
}

void loop() {}   // tidak pernah dieksekusi pada pola Deep Sleep murni
```

4. Skenario uji dan hasil yang diharapkan

Skenario	Data masukan	Hasil yang diharapkan
Cuaca stabil	Riwayat: 1012,0; 1011,8; 1011,7 (tiap 15 menit); sekarang 1011,5	Penurunan = 1012,0 − 1011,5 = 0,5 hPa < 3 → NORMAL, tidur lagi
Tekanan jatuh drastis	Riwayat 45, 30, dan 15 menit lalu: 1009,8; 1008,9; 1007,9; sekarang 1006,6	Penurunan = 1009,8 − 1006,6 = 3,2 hPa ≥ 3 → DARURAT (LED, buzzer, JSON ALERT)
Getaran mendadak	Bangun EXT0 saat jadwal belum tiba; tekanan normal	Status WASPADA; jadwal 15 menit tetap dipertahankan (sisa waktu dihitung ulang)
Getaran + tekanan jatuh	EXT0 dan penurunan ≥ 3 hPa	DARURAT (prioritas lebih tinggi dari WASPADA)

Skenario	Data masukan	Hasil yang diharapkan
Sensor lepas	BME280 tidak terdeteksi	JSON error, node kembali tidur tanpa mengubah riwayat RTC

5. Catatan dan batasan

- alarmBunyi() memakai delay() yang dapat diterima karena node aktif singkat; versi lanjut dapat memakai task FreeRTOS.
- GPIO 15 adalah strapping pin; pastikan level sensor getar saat boot tidak mengganggu proses booting. Alternatif: pindah ke RTC GPIO lain (mis. GPIO 32/33/34/35).
- Waktu RTC (gettimeofday) tetap berjalan saat Deep Sleep tetapi tereset saat power-on reset; riwayat RTC juga hilang pada kondisi yang sama, sehingga keduanya konsisten.
- Pengembangan lanjutan: kirim peringatan via MQTT/LoRa, simpan riwayat ke flash (NVS) agar bertahan saat baterai habis, dan pakai ambang adaptif.